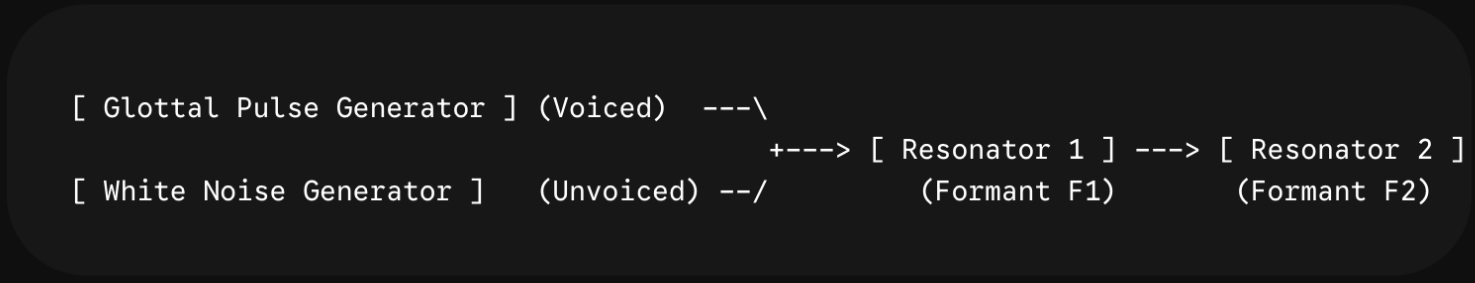
The non-sine-based voice engine in your source code is a classic implementation of **Formant Synthesis using a Cascade Resonator Model**, heavily inspired by early speech engines like Dennis Klatt's 1980 design.

Instead of stacking individual geometric sine wave tracks to mimic harmonics, it uses digital signal processing blocks to mimic the physical acoustics of the human vocal tract.

1. The Acoustic Simulation Blueprint

The engine splits sound production into two distinct parts: a **Sound Source** and a **Resonant Filter**.



A. The Excitation Source (The "Lungs and Vocal Chords")

Depending on the token being evaluated, the source generates one of two wave types:

- **Voiced Sounds (Vowels/Nasals):** The code calls `get_glottal_pulse(hybrid_p0)`. This isn't a plain geometric oscillator; it mathematically approximates the pressure spike created when human vocal cords flap open and shut. It yields a rich harmonic sawtooth wave that provides the fundamental pitch (F_0).
- **Unvoiced Sounds (Fricatives/Plosives like S, SH, T):** The engine switches to a linear congruential random number generator (`noise_seed * 1103515245 + 12345`) passed through a simple low-pass filter to generate white/pink noise, simulating turbulent air rushing past the teeth or lips.

B. The Resonator Chain (The "Vocal Tract")

The raw source signal is fed into a cascaded chain of three **Resonator** objects (`R1` , `R2` , and `R3`).

- Each `Resonator` is a digital **two-pole bandpass filter**.

- The code computes specific filter coefficients (a, b, c) using the targeted center frequencies (F_1, F_2, F_3) and bandwidths (B_1, B_2, B_3) defined in your `PHONEMES` map.
- As the raw glottal pulse passes through the filter chain, the resonators suppress most frequencies while amplifying the formant peaks, shaping the buzzy raw input into distinct phonetic syllables like "EH", "IY", or "OW".

2. Main Structural Advantages Over Sines

Feature	Sine-Wave Synthesis	Resonator Cascade Synthesis
Spectral Profile	Sharp, isolated sine-wave spikes. Can sound glassy, alien, or thin.	Rich, continuous spectrum with natural acoustic decay valleys.
Noise Integration	Hard to integrate smoothly; must use specialized plosive sine burst curves.	Intuitively shapes noise spectra by pushing randomness directly through the filter bank.
Computational Style	Additive Fourier modeling (building from scratch).	Subtractive structural modeling (shaping a raw source).

It is **100% created dynamically** in real time—there are absolutely no pre-recorded audio samples, WAV files, or data tables of recorded sound hidden inside the engine.

This approach is known as **pure mathematical (or algorithmic) synthesis**.

Every single sample of audio written into the output vector is calculated on the fly using explicit mathematical formulas. Here is how the engine builds everything from scratch without relying on samples:

1. The Glottal Pulse is a Pure Mathematical Curve

Instead of playing back a recording of vocal cords vibrating, the engine uses an explicit piece-wise function in `get_glottal_pulse()` to draw the wave shape sample-by-sample:

$$f(\phi) = 3\left(\frac{\phi}{0.22}\right)^2 - 2\left(\frac{\phi}{0.22}\right)^3$$

By changing the phase variable (ϕ) based on `current_f0`, the engine alters the pitch dynamically.

2. The Noise is Algebriacally Generated

To get the turbulent air sound for consonants like "S" or "SH", the engine uses a **Linear Congruential Generator (LCG)**:

C++

```
noise_seed = noise_seed * 1103515245 + 12345;
```

This formula intentionally overflows a 32-bit integer to generate pseudo-random numbers on the fly, mimicking white noise without needing a static noise sample asset.

3. The Filters use Time-Domain Difference Equations

When the code shapes the sound using `R1.process()`, it isn't matching the sound against a database. It runs a live **Infinite Impulse Response (IIR)** mathematical filter loop:

C++

```
double out = a * in + b * y1 + c * y2;
```

This equation takes the current raw audio sample, modifies it using the two previous output samples (`y1` and `y2`), and dynamically recalculates the acoustic resonance thousands of times per second.